SC4024 Data Visualisation

NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-30 — Generated for bumbsclaw/ntu-cs-textbooks

Contents

Chapter 1

Visualisation Foundations & Perception

> *"A picture is worth a thousand numbers."* — But only if the picture is honest, legible, and matched to how eyes and brains actually work. This chapter asks what visualisation is and why perception sets its limits.

From zero: we build here, no prior visualisation needed. We assume only sets, numbers, and curiosity about pictures. Every notion—data table, visual channel, pre-attentive pop-out—is motivated by intuition, then defined, then exercised. No prior bachelor's knowledge is required.

Learning Outcomes

After this chapter you will be able to:

(L1) define data visualisation and distinguish exploratory, explanatory, and presentational goals;

(L2) explain the human visual pipeline from retina to cortex and pre-attentive processing;

(L3) state Weber–Fechner and Stevens laws and predict perceived magnitude;

(L4) rank visual channels by effectiveness for quantitative, ordinal, and nominal data;

(L5) apply the Munzner nested model to characterise a visualisation design.

1.1 What Is Data Visualisation?

Intuition. Anscombe's quartet is four datasets with identical means, variances, and correlations—yet their scatterplots look utterly different. Numbers hide structure that pictures reveal. Visualisation is the deliberate translation of data into a picture so that structure becomes visible without calculation.

Formal view. A dataset is a collection of *items* with *attributes* (columns). A visualisation specifies a mapping $f : \text{data} \to \text{marks} \times \text{channels}$ rendered as an image, optionally interactive. Card, Mackinlay, and Shneiderman's

reference model refines this into a pipeline: raw data → data tables → visual structures → views. Each stage transforms representation; each choice can help or mislead.

Definition 1.1 (Visualisation pipeline)**.** *Data transformation* cleans and derives data. *Visual mapping* assigns data attributes to *marks* (point, line, area, volume) and *visual channels* (position, length, angle, colour, size, shape). *View transformation* maps the visual structure to screen coordinates, handling zoom, pan, and clipping.

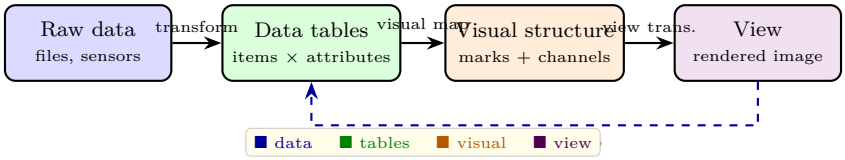

Figure 1.1: The Card–Mackinlay–Shneiderman visualisation pipeline. Interaction can feed back to any earlier stage; the forward path is data → picture.

Three goals orient design: *explore* (find unknown patterns; analyst does not know what is there), *explain* (communicate a known finding; audience must grasp it), and *present* (enable repeated lookup; dashboards). The same data needs different pictures for different goals.

1.2 How Eyes See: The Visual System

Intuition. Look at a field of blue dots with one red dot. The red dot leaps out instantly—you do not scan dot by dot. That leap is *pre-attentive* processing: before conscious attention, the visual cortex flags certain features in parallel across the whole field.

Formal view. Light hits photoreceptors (rods and cones), retinal ganglion cells extract contrast, LGN relays to primary visual cortex V1 which detects edges and orientation, then higher areas separate form, colour, and motion. Pre-attentive features include hue, intensity, size, orientation, and motion. Conjunctions (e.g. red vertical among red horizontal and blue vertical) require serial search and are slow.

Definition 1.2 (Weber–Fechner and Stevens)**.** Weber's law: just-noticeable difference $\Delta I/I = k$ (constant). Fechner's law integrates this: perceived sensation $S = k \log I$. Stevens' power law refines it: $S = kI^n$ where n depends on modality ($n \approx 0.33$ for brightness, ≈ 0.7 for length, ≈ 1 for line length judged naively, > 1 for electric shock).

These laws matter: if perception is logarithmic, linear encodings of brightness will be misread; if length is underestimated, bar charts need careful baselines.

1.3 Visual Channels and Effectiveness

Intuition. To show a number you can make a bar longer, a dot higher, a circle bigger, or a patch darker. Some of these are effortlessly precise (position), others are vague (colour saturation). Choosing the channel determines whether the reader gets the answer in a glance or a guess.

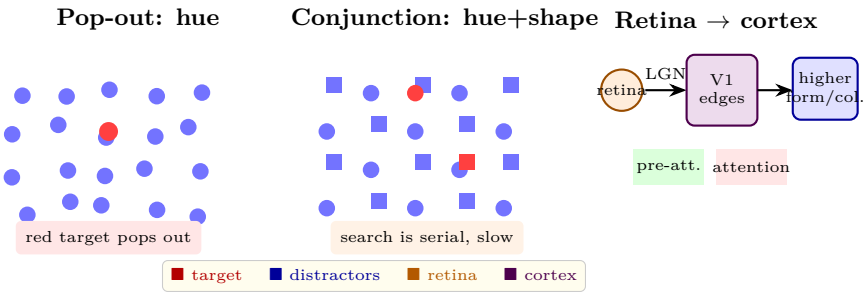

Figure 1.2: Pre-attentive processing: hue pops out in parallel (left); conjunctions of hue and shape require serial search (centre). Right: schematic visual pipeline.

Definition 1.3 (Marks and channels after Bertin/Mackinlay)**.** A *mark* is a geometric primitive: point, line, area. A *channel* controls a mark's appearance: position (x, y), length, angle, area, volume, colour hue/saturation/luminance, shape, texture, motion. Data attributes are typed as quantitative (ordered, arithmetic), ordinal (ordered, no arithmetic), or nominal (categorical). A channel's *expressiveness* is whether it can encode the attribute type without implying false properties; its *effectiveness* is perceptual accuracy.

Mackinlay's ranking (validated by Cleveland–McGill) orders channels by quantitative judgment accuracy: position on common scale $>$ length $>$ angle/slope $>$ area $>$ volume $>$ colour. For nominal data, hue and position dominate; length misleadingly implies order and magnitude.

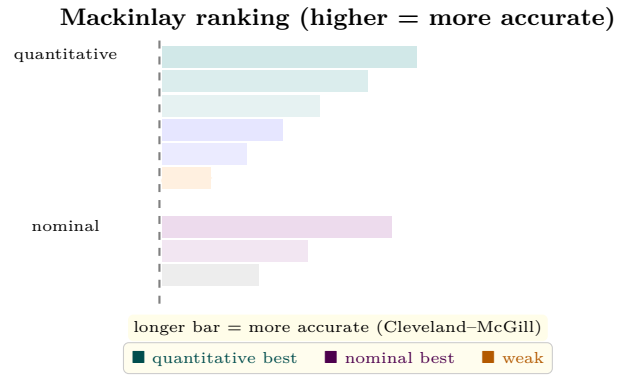

Figure 1.3: Effectiveness ranking of channels. Position dominates for quantitative data; hue and position dominate for nominal. Colour luminance is poor for precise quantity.

Example 1.1 (Anscombe with four pictures)**.** Four datasets share $\bar{x} = 9$, $\bar{y} = 7.5$, correlation ≈ 0.82, regression $y = 3 + 0.5x$. Plotted, one is linear, one is curved, one has an outlier, one is a vertical line with one far point. A table of summary statistics cannot distinguish them; a scatterplot can. This motivates the entire course.

Listing 1.1: Anscombe's quartet: same stats, four pictures.

```
import matplotlib.pyplot as plt

# Anscombe quartet (first few rows shown; full 11 rows per set)
anscombe = {
  "I":  ([10,8,13,9,11,14,6,4,12,7,5],
         [8.04,6.95,7.58,8.81,8.33,9.96,7.24,4.26,10.84,4.82,5.68]),
  "II": ([10,8,13,9,11,14,6,4,12,7,5],
         [9.14,8.14,8.74,8.77,9.26,8.10,6.13,3.10,9.13,7.26,4.74]),
}
```

```
fig, axes = plt.subplots(1, 2, figsize=(8,3))
for ax, (k,(xs,ys)) in zip(axes, anscombe.items()):
    ax.scatter(xs, ys, c="steelblue", edgecolor="white")
    ax.set_title(k); ax.set_xlim(2,16); ax.set_ylim(2,12)
plt.tight_layout(); plt.savefig("anscombe.pdf")
```

Listing 1.2: Pre-attentive intuition and Stevens law sketch.

```
import numpy as np, matplotlib.pyplot as plt

# Stevens power law: perceived = k * I^n
I = np.linspace(0.1, 10, 200)
for n, label in [(0.33,"brightness"), (0.7,"length"), (1.0,"line")]:
    plt.plot(I, I**n, label=f"n={n} {label}")
plt.xlabel("physical intensity I"); plt.ylabel("perceived S")
plt.legend(); plt.title("Stevens power law S=kI^n")
plt.savefig("stevens.pdf")
```

1.4 The Nested Model of Design

Intuition. A visualisation can fail at many levels: the wrong question, the wrong data abstraction, the wrong picture, or the wrong pixels. Tamara Munzner's nested model stacks these choices so a failure at an outer layer cannot be rescued by polishing an inner one.

Four nested levels: *domain situation* (who are the users, what do they need?), *data/task abstraction* (what data and tasks, precisely?), *visual encoding/interaction* (what marks, channels, and interactions?), *algorithm* (how to compute it fast and correctly?). Validation flows outward: algorithm benchmarks, encoding studies, abstraction interviews, domain field studies.

Remark 1.1 (Why this book follows intuition→formal→example). Each chapter mirrors the nested model: we start with the human and the question (domain), abstract the data, choose the picture, then show how to build it. Exercises ask you to move one layer outward and re-validate.

1.5 Why Perception Constrains Design

Even a correct pipeline fails if it asks eyes to do what they cannot. Two rules follow: never encode precise quantities in colour saturation alone, and never rely on angle or area when position would suffice. Interactive highlighting can rescue a weak channel by recruiting pre-attentive pop-out (e.g. hover turns a faint line red), but it cannot fix a fundamentally mismatched mapping. The next chapter turns these constraints into a systematic vocabulary.

1.6 Chapter Notes

Visualisation as a field crystallised in the 1980s–90s (Bertin 1967 Semiology of Graphics; Card et al. 1999 Readings in Information Visualization). Mackinlay's APT (1986) automated channel choice; Cleveland and

McGill (1984) measured channel accuracy; Munzner (2014) unified design as the nested model. Tufte (1983) and Ware (2020) connected design honesty to perception. The next chapter formalises data types and the channel vocabulary introduced here.

1.7 Exercises

E1.1 **Anscombe extended.** Compute $\bar{x}, \bar{y}$, variance, and Pearson r for Anscombe set III by hand for 5 points and verify equality across sets. Plot all four sets and describe in one sentence how each would mislead a model fitted only to summaries.

E1.2 **Stevens fitting.** Given perceived brightness judgments $S = [2, 3.6, 5.8]$ at intensities $I = [1, 4, 9]$, fit $S = kI^n$ in log-log space (linear regression of $\log S$ on $\log I$). Report n and interpret whether brightness is compressive ($n < 1$).

E1.3 **Channel ranking task.** For the task "compare GDP of 6 countries to within 10% accuracy" propose three encodings (position, area, colour luminance) and rank them using Mackinlay/Cleveland–McGill. Justify with a sketch and one-sentence effectiveness argument per channel.

E1.4 **Nested-model mapping.** Take the visualisation "COVID-19 dashboard with map + time series + hospital bar chart." Assign each component to a nested-model level, state one threat to validity per level, and propose one validation method per threat.

E1.5 **Misleading chart critique.** Find a chart with a truncated y-axis (or 3-D extrusion) online; redesign it with a zero baseline (or 2-D bars) and explain the perceptual error the original induces using Weber–Fechner or Stevens reasoning.

Chapter 2

Data Types & Visual Encodings

"You cannot encode what you have not understood." — Before choosing a chart, name what kind of thing each column is and what kind of thing the table is. The encoding follows from that naming.

From zero: we build here, no prior data-typing needed. We assume only tables of values. Every data type, mark, and channel is introduced with intuition before formalism, with small examples and code. No prior visualisation maturity is required.

Learning Outcomes

After this chapter you will be able to:

(L1) classify attributes as nominal, ordinal, quantitative, temporal, and spatial;

(L2) distinguish dataset types: tables, networks, fields, geometry, and hierarchies;

(L3) define marks (point, line, area) and channels and assign them to data types;

(L4) apply expressiveness and effectiveness to critique and fix an encoding;

(L5) encode uncertainty and missing data without implying false precision.

2.1 Data Types: WhatKind of Thing Is It?

Intuition. A column called "country" (Singapore, France, Kenya) is not a number—you cannot average it, and sorting it is arbitrary. A column called "temperature" is a number where averaging and subtracting mean something. A column called "shirt size" (S < M < L) has order but no meaningful subtraction. If you give a nominal column to a channel that implies magnitude, you lie.

Formal view. Stevens' typology: *nominal* (categories, only = test), *ordinal* (ordered, <, no arithmetic), *quantitative* (continuous or discrete, arithmetic meaningful; interval vs ratio depending on whether zero is

absolute). Temporal is ordered quantitative with calendrical structure; spatial is inherently 2- or 3-dimensional. Hierarchical or network relations are not attributes but *structures* on items. Keys identify items; values are measured on them.

Definition 2.1 (Attribute and dataset types)**.** An *attribute* maps items → values of a type: nominal, ordinal, quantitative (interval/ratio), temporal, spatial. A *dataset type* describes the whole collection: *table* (items × attributes), *network* (nodes + links + attributes), *field* (continuous domain, e.g. temperature over space), *geometry* (shape: points, lines, polygons), *hierarchy/tree*, *cluster/set*.

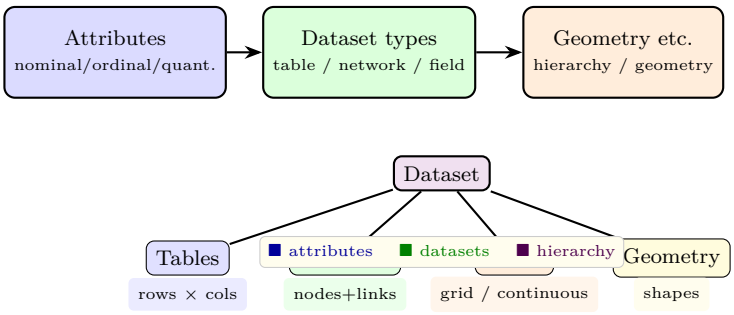

Figure 2.1: From attribute types to dataset types. The encoding choice depends on both: a table likes position and bars; a field likes colour and contours; a network likes node-link or matrix.

Key idea: *keys* vs *values*. A table has independent keys (e.g. country, year) and dependent values (e.g. GDP). Visual channels for keys should be spatial (position) to show the lookup; values use retinal channels (colour, size, length).

2.2 Marks and Channels

Intuition. Every picture is made of primitive marks: a dot at (x, y) for a scatterplot, a line through points for a time series, an area for a bar or polygon. How each mark looks—its position, length, colour, size—encodes data. Position is the richest: it can carry any type precisely. Colour hue is good for categories; saturation and luminance are weaker for numbers.

Definition 2.2 (Marks and channels)**.** A *mark* is a geometric primitive: *point* (0-D, dot), *line* (1-D, stroke), *area* (2-D, fill). A *channel* controls a mark's appearance: position (x, y), length/height, angle, area, volume, colour hue/saturation/luminance, shape, tilt, texture, motion. Each channel has *type compatibility*: position → any type; length → quantitative/ordinal; hue → nominal; saturation/luminance → ordinal/quantitative (weak).

Scales and mappings. An attribute domain (e.g. temperature $[-20, 40]$) maps via a *scale* to a channel range (e.g. luminance $[10\%, 90\%]$ or position $[0, \text{width}]$). Scales may be linear, logarithmic, or custom. Reversed or truncated scales can lie; nominal attributes need categorical (non-ordered) palettes. Always expose the scale: a missing legend turns a picture into a riddle.

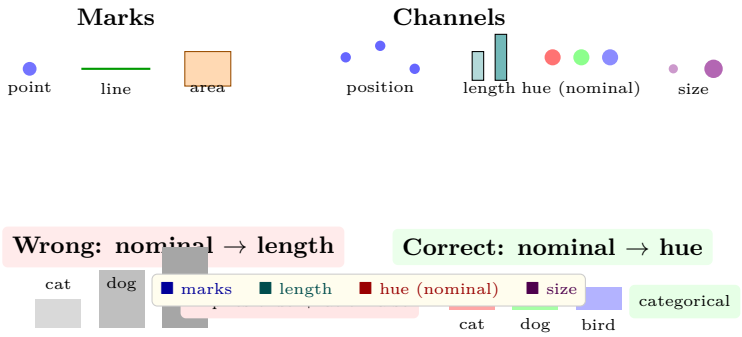

Figure 2.2: Marks (point, line, area) and channels (position, length, hue, size). Bottom: expressiveness mismatch—length implies order and magnitude, so it is wrong for nominal data; hue is correct.

2.3 Expressiveness, Effectiveness, and Uncertainty

Two tests decide a mapping. *Expressiveness*: does the channel say only truths about the data type? (Hue says "different" but not "bigger"—good for nominal, bad for quantitative.) *Effectiveness*: how accurately do eyes read it? (Position > length > angle > area > colour for quantity.)

Example 2.1 (Fixing a mismatch)**.** A bar chart that colours countries by GDP uses nominal hue for a quantitative attribute—inexpressive? Actually quantifying colour's luminance is weak; better to use bar length or position. Conversely, mapping "continent" to bar length would claim Europe > Asia, which is nonsense. Fix: swap channels so quantitative → length/position, nominal → hue.

Uncertainty needs its own channel. *Value-suppressing* palettes fade colour as uncertainty rises; error bars and bands attach to quantitative position encodings. Missing data should not be interpolated silently: show a gap, a hatched mark, or an explicit "no data" glyph, otherwise the eye assumes zero.

Figure 2.3: Encoding uncertainty: value-suppressing palettes fade with uncertainty (left); missing data is shown as a gap or hatched region (right), not as zero.

Listing 2.1: Altair encodings: binding data types to channels with scales.

```
import altair as alt, pandas as pd

df = pd.DataFrame({"country":["SG","FR","KE"],"gdp":[90,42,6],
                   "continent":["Asia","Europe","Africa"]})
# quantitative -> position+length (bar), nominal -> hue
chart = alt.Chart(df).mark_bar().encode(
    x=alt.X("gdp:Q", scale=alt.Scale(zero=True), title="GDP (quant.)"),
    y=alt.Y("country:N", sort="-x"),
    color=alt.Color("continent:N", legend=alt.Legend(title="Nominal"))
).properties(title="Expressive: Q->length, N->hue")
chart  # in a notebook this renders; .to_json() gives Vega-Lite
```

Listing 2.2: Value-suppressing sketch and missing-data gaps with Matplotlib.

```python
import numpy as np, matplotlib.pyplot as plt

x = np.linspace(0, 10, 50)
y = np.sin(x)
uncert = np.linspace(0, 1, 50)          # 0=certain, 1=uncertain
# value-suppressing: alpha fades with uncertainty
plt.figure(figsize=(6,2.5))
for i in range(len(x)-1):
    a = 1 - 0.85*uncert[i]              # fade
    plt.plot(x[i:i+2], y[i:i+2], color="steelblue", alpha=a, lw=2.5)
plt.title("Value-suppressing: certainty -> opacity"); plt.tight_layout()
plt.savefig("suppress.pdf")

# missing data: break the line
y2 = y.copy(); y2[20:28] = np.nan        # gap = missing
plt.figure(); plt.plot(x, y2, color="orange"); plt.title("Gap = missing")
plt.savefig("missing.pdf")
```

2.4 Composing Encodings

Two attributes can share a view by *composition*: superimposed on the same marks (colour + position), juxtaposed in small multiples, or layered with transparency. Small multiples keep each encoding simple and let position do the heavy lifting; superposition saves space but risks overplotting, rescued by opacity or aggregation. The choice is again an expressiveness trade-off: does the composition imply a false relationship?

2.5 Chapter Notes

Stevens (1946) typed scales; Bertin (1967) catalogued marks and channels; Mackinlay (1986) formalised expressiveness/effectiveness as automatic design criteria; Munzner (2014) unified attribute and dataset taxonomies; Correll (2018) systematised uncertainty encodings. The next chapter turns "what to encode" into "how to design it well" via colour and composition.

2.6 Exercises

E2.1 **Classify.** For the dataset {student ID, final grade (A–F), essay score 0–100, submission date}, label each attribute nominal/ordinal/quantitative/temporal and justify. Propose one expressive channel per attribute.

E2.2 **Channel critique.** A pie chart shows GDP of 8 countries as slice angles. Using Mackinlay/Cleveland–McGill, argue why a sorted bar chart (length + position) would be more effective for "which is larger by how much?" Sketch both and annotate the channel swap.

E2.3 **Expressiveness fix.** A designer maps "blood group (A/B/AB/O)" to bar length and "haemoglobin

(g/dL)" to hue. Explain both mismatches and redesign the two encodings so each channel matches its data type.

E2.4 **Uncertainty.** You have temperature estimates with 95% intervals. Design two alternatives: (a) error bars on a line chart, (b) value-suppressing band. When does each fail, and how would you show a sensor outage (missing run)?

E2.5 **Altair build.** Using Altair, encode a table with one nominal, one ordinal (shirt size S<M<L<XL), and one quantitative attribute in a single chart without mismatch. Include a Vega-Lite JSON scale that maps ordinal to ordered colour luminance and explain why a rainbow hue would be wrong.

Chapter 3

Design Principles & Colour Theory

> *"Good visualisation is not decoration—it is argument made visible."* — Every inked pixel should earn its keep, and colour should help seeing, not merely please the eye.

From zero: we build here, no prior design needed. We start from what ink does on a page and how eyes group it, then formalise design rules, colour spaces, and palettes, with before/after fixes and code. No prior art training is required.

Learning Outcomes

After this chapter you will be able to:

(L1) apply data-ink ratio, lie factor, and chartjunk critiques to a chart;

(L2) use Gestalt laws (proximity, similarity, continuity, closure) to guide layout;

(L3) distinguish colour spaces RGB, HSL, Lab/HCL and choose the right one;

(L4) design sequential, diverging, and qualitative palettes and check CVD safety;

(L5) redesign a flawed chart with annotation, hierarchy, and colour fixes.

3.1 Design Honesty: Ink, Lies, and Clutter

Intuition. Imagine a bar chart where most ink is gridlines, 3-D bevels, and a pictorial icon whose height and area tell different stories. The data is a minority of the ink and the eye is misled. Edward Tufte called the remedy the *data-ink ratio*: maximise ink that encodes data, erase the rest.

Formal view. Data-ink ratio = (ink for data)/(total ink). *Chartjunk* is ink that conveys no information (or worse, distracts). The *lie factor* = (size of effect in graphic)/(size of effect in data) should be 1; a bar whose

length doubles while data rises 10% has lie factor 20, a gross distortion. *Data density* is data per unit area; higher is better provided legibility remains.

Definition 3.1 (Tufte principles)**.** *Maximise data-ink* within reason: remove borders, redundant ticks, and decoration that repeats what position already says. *Avoid chartjunk*: no 3-D extrusion on 2-D data, no gratuitous icons. *Lie factor* ≈ 1: encodings must be proportional to data, with zero baselines for length/area. *Layer and separate*: use thin gridlines behind data, not over it.

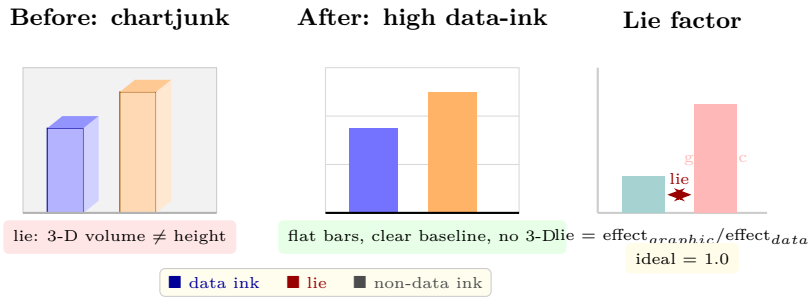

Figure 3.1: Tufte's principles: remove chartjunk and 3-D distortion (left → centre); keep the lie factor at 1 (right). The repaired chart uses position and length honestly.

3.2 Gestalt: How Eyes Group

Intuition. Place six dots in two tight clusters and you see two groups without being told. Colour three of them red and the rest blue and you see colour groups. The visual system groups by *Gestalt laws* before conscious reasoning—design can recruit this or fight it.

Definition 3.2 (Gestalt laws for layout)**.** *Proximity*: nearby elements group. *Similarity*: similar colour/shape groups. *Continuity/good continuation*: aligned elements read as a path. *Closure*: incomplete contours are perceived as closed. *Figure–ground*: one region reads as figure, the rest as ground. *Common fate*: elements moving together group.

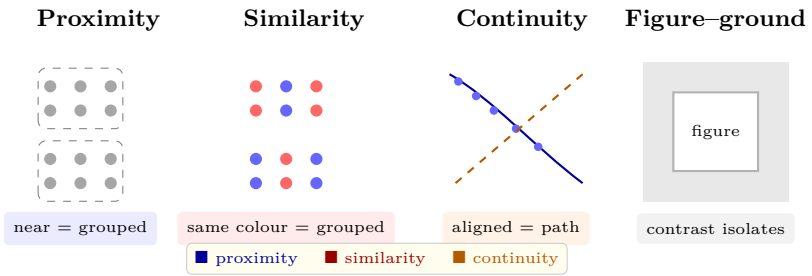

Figure 3.2: Gestalt laws: proximity, similarity (colour), continuity, and figure–ground. Good layout places related items near, colours them alike, and aligns them.

Practical rules: put related charts near each other, colour related series alike, align axes, and close a layout with a border or background so it reads as one figure.

3.3 Colour: Spaces and Palettes

Intuition. The rainbow looks uniform but is not: yellow and cyan appear brighter and more distinct than deep blue, so a rainbow colormap makes yellows shout even where data is flat. RGB, the monitor's language, is also non-uniform. For faithful visualisation we need a space where equal steps in numbers give equal steps in perception.

Definition 3.3 (Colour spaces)**.** *RGB*: additive red-green-blue cube (device-dependent, not perceptually uniform). *HSL/HSV*: hue–saturation–lightness/value (intuitive but not uniform). *CIE Lab*: L luminance, a green–red, b blue–yellow, designed so Euclidean distance ΔE approximates perceived difference. *HCL*: polar Lab (hue, chroma, luminance)—intuitive and approximately uniform; preferred for palettes.

Palette types: *sequential* (one hue, luminance ramp; for ordered/quantitative from low to high), *diverging* (two hues with neutral midpoint; for deviation from centre), *qualitative* (distinct hues at similar luminance/chroma; for nominal). ColorBrewer codifies these; append CVD (colour-vision deficiency) simulation to verify.

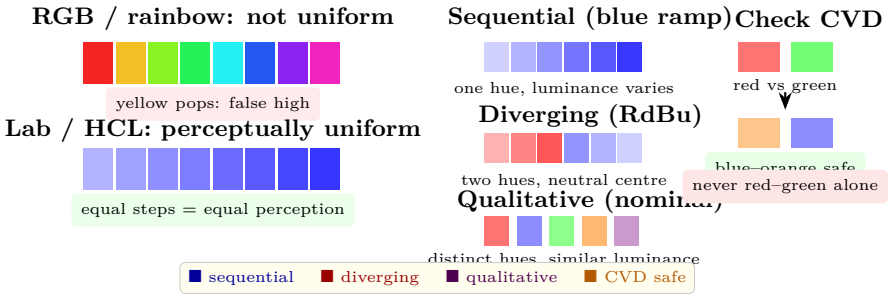

Figure 3.3: Colour: rainbow RGB is not perceptually uniform (top left); Lab/HCL ramps are (centre left). Right: sequential, diverging, and qualitative palettes, plus the red–green CVD pitfall and its blue–orange remedy.

Listing 3.1: Building honest palettes: sequential, diverging, qualitative, and CVD check.

```
import matplotlib.pyplot as plt, numpy as np
from matplotlib.colors import LinearSegmentedColormap

# Sequential: single-hue luminance ramp (perceptually ordered)
seq = LinearSegmentedColormap.from_list("seq", ["#f7fbff","#08306b"])
# Diverging: two hues with neutral centre
div = LinearSegmentedColormap.from_list("div", ["#b2182b","#f7f7f7","#2166ac"])
# Qualitative: distinct hues, similar luminance
qual = ["#e41a1c","#377eb8","#4daf4a","#984ea3","#ff7f00"]
# demo: apply sequential to a gradient
vals = np.linspace(0, 1, 10)
fig, ax = plt.subplots(figsize=(6,1.2))
ax.imshow([vals], cmap=seq, aspect="auto"); ax.set_title("Sequential blue ramp (ordered)")
ax.set_axis_off(); plt.savefig("seq.pdf")
# CVD note: test with colorspacious or simulate deuteranopia; avoid red-green alone
```

Listing 3.2: Lie-factor audit and data-ink sketch.

```
# Lie factor = size_of_effect_graphic / size_of_effect_data
def lie_factor(data_old, data_new, graphic_old, graphic_new):
    return (graphic_new-graphic_old)/graphic_old / ((data_new-data_old)/data_old)
```

```
4
5 print(lie_factor(100, 110, 2.0, 4.0))  # 10% data, 100% graphic -> lie=10 (dishonest)
6 print(lie_factor(100, 110, 2.0, 2.2))  # honest: both +10% -> lie=1.0
7
8 # Data-ink: count non-data pixels (grid, border) vs data pixels in a mock
9 # In practice: remove border, lighten grid to 0.3 alpha, drop 3-D
```

3.4 Before/After: A Redesign

Intuition. A flawed chart: rainbow background, 3-D bars, truncated y-axis at 50, legend far from data, tiny labels. Fix sequence: restore zero baseline (lie factor to 1), flatten to 2-D, replace rainbow with single-hue sequential, move legend beside data (proximity), align and enlarge labels, add one annotation that states the takeaway. Each fix maps to a principle above.

3.5 Chapter Notes

Tufte (1983, 1990) defined data-ink, lie factor, and chartjunk; Ware (2020) grounded them in perception; Brewer (ColorBrewer) systematised palettes; Stone et al. and Wong (2011) codified CVD-safe design; Munzner (2014) framed honesty as across-level validation. The next chapter applies these rules to statistical charts and interaction.

3.6 Exercises

E3.1 **Lie factor.** A company's profit rose from \$80M to \$88M (+10%) but its bar grew from 2 cm to 3 cm (+50%). Compute the lie factor. Redraw with a zero baseline and report the new factor.

E3.2 **Chartjunk audit.** Take a chart with heavy grid, border, and background shading. Estimate data-ink ratio before and after removing non-data ink. Which removals most improved the ratio without harming legibility?

E3.3 **Palette design.** Design three palettes for the same 7-category nominal, 9-step sequential, and ± diverging tasks. Show swatches and argue hue/luminance choices. Simulate deuteranopia (e.g. via colorspacious) and revise any failing pair.

E3.4 **Gestalt redesign.** A dashboard scatters 6 unrelated charts with equal spacing and random colours. Use proximity, similarity, and alignment to regroup them into 2 tasks × 3 views. Sketch before/after and label each Gestalt law applied.

E3.5 **Lab vs RGB interpolation.** Interpolate between blue and yellow in RGB and in Lab (via `colorspacious` or Altair's HCL). Plot the midpoint swatches and explain why the RGB midpoint looks muddy grey while Lab stays vivid and uniform.

Chapter 4

Statistical Charts & Interaction

> *"Overview first, zoom and filter, then details on demand."* — Shneiderman's mantra captures the move from static distributions to interactive exploration. This chapter builds that bridge.

From zero: we build here, no prior statistics beyond means needed. We introduce histograms, KDE, boxplots, and scatter matrices from intuition, then add brushing, linking, and dynamic queries, with small examples and code. No prior stats-visualisation experience is required.

Learning Outcomes

After this chapter you will be able to:

(L1) choose among histogram, KDE, boxplot, violin, and rug for a distribution task;

(L2) select bin width and bandwidth and explain their bias–variance trade-off;

(L3) design scatterplot matrices and parallel coordinates for multivariate data;

(L4) implement brushing, linking, filtering, and details-on-demand;

(L5) evaluate interaction latency and visual scalability limits.

4.1 Showing One Distribution

Intuition. You have 200 exam scores. A list says little; a picture of how many fall in each interval says a lot. But the picture depends on how you slice the intervals—too wide and you hide bimodality, too narrow and noise dominates.

Formal view. A *histogram* partitions the range into bins of width h and counts n_j per bin; bar height is n_j (or density $n_j/(nh)$). A *kernel density estimate* smooths: $\hat{f}_h(x) = \frac{1}{nh}\sum K((x - x_i)/h)$ for kernel K (Gaussian).

Histogram is a box kernel on bin centres; KDE is its smooth limit. A *boxplot* shows median, quartiles Q_1, Q_3, IQR, whiskers to 1.5 IQR, and outliers. *Violin* adds a mirrored KDE; *rug* marks each point.

Definition 4.1 (Bin and bandwidth rules)**.** Heuristics: Sturges $k = \lceil \log_2 n + 1 \rceil$ bins; Scott $h = 3.5\sigma n^{-1/3}$; Freedman–Diaconis $h = 2\,\mathrm{IQR}\, n^{-1/3}$ (robust to outliers). For KDE, bandwidth h trades bias (oversmooth, hides peaks) vs variance (undersmooth, wiggles). Leave-one-out cross-validation can tune h.

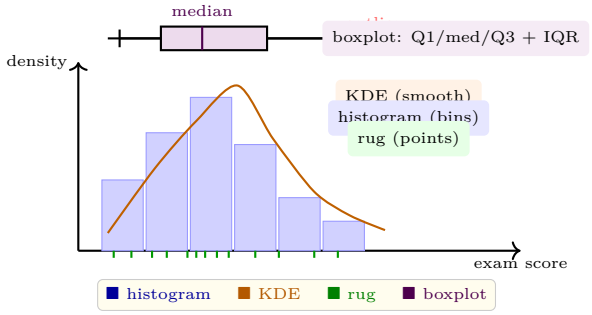

Figure 4.1: One distribution, four views: histogram (bars), KDE (smooth orange line), rug (ticks), and boxplot (top). Bin width and bandwidth control bias vs variance.

Bias–variance in pictures. Wider bins / larger h lower variance but blur true modes (bias); narrower bins show sampling noise (variance). Overlay all three—bars, KDE, rug—when in doubt: the rug keeps you honest.

4.2 Showing Many Variables

Intuition. Two variables invite a scatterplot; three invite colour or size; ten invite a wall of views. The challenge is not drawing but *comparing*: can eyes compare distributions across groups and relationships across pairs?

Two families: *scatterplot matrix (SPLOM)* shows every pair as a small scatterplot—good for up to ~ 8 variables, then clutter dominates. *Parallel coordinates* draw each variable as a vertical axis and each item as a polyline crossing them—good for spotting clusters and outliers across many dimensions, weakened by axis ordering and overplotting (rescued by bundling, opacity, and reordering).

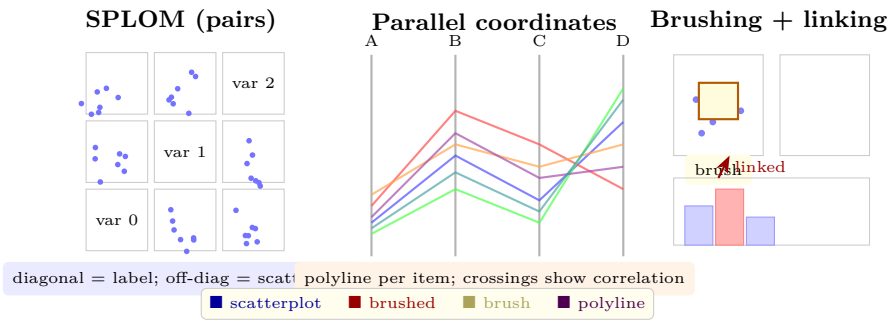

Figure 4.2: Multivariate: scatterplot matrix (left) shows all pairs; parallel coordinates (centre) show all variables as axes; brushing a scatterplot highlights linked points in a histogram (right).

4.3 Interaction: From Picture to Dialogue

Intuition. A static chart answers one question. An interactive chart lets the reader ask the next one: "what if I filter to 2023?" "who are those outliers?" Interaction turns the pipeline into a loop where the reader steers data transforms and view parameters directly.

Definition 4.2 (Shneiderman mantra and interactions)**.** *Overview first, zoom and filter, then details on demand.* Primitive interactions: *navigate* (pan/zoom), *select* (brush: rectangular, lasso), *filter* (sliders, checkboxes), *link* (selection in one view highlights same items elsewhere), *aggregate* (change binning/level of detail), *annotate* (tooltip, label on hover). *Dynamic queries* update views continuously as sliders move (latency target < 100 ms).

Example 4.1 (Brushing and linking)**.** A scatterplot of price vs mileage and a histogram of price share the same items. Dragging a rectangle (brush) in the scatterplot selects points; the histogram re-colours its bins to show the selection's price distribution. Two views, one selection, instant feedback. Without linking, the reader must memorise and cross-reference.

Remark 4.1 (Linking as coordination). Linking is a coordination of views, not just two charts side by side. The selection is a shared data predicate; every view that listens to it updates. This is why Altair/D3 model selections as first-class objects rather than per-chart state.

Latency matters. > 500 ms breaks flow; < 100 ms feels direct. Scalability tricks: aggregate before rendering (bin, sample, or summarise), use canvas/WebGL for many marks, debounce slider events, and precompute indexed selections.

4.4 Visual Scalability

Even with fast interaction, eyes have limits. Overplotting hides structure once marks exceed ∼10k visible points; aggregation (hexbin, 2-D histogram, contour) trades detail for legibility. For high-cardinality nominal splits, small multiples beat colour overload (eyes hold ∼7 colours in working memory). When rows reach millions, server-side binning with client-side brushing keeps latency flat; the picture shows summaries, the interaction reveals individuals on demand.

Listing 4.1: Histogram vs KDE and bandwidth choice with NumPy/Matplotlib.

```
import numpy as np, matplotlib.pyplot as plt

np.random.seed(0)
x = np.concatenate([np.random.normal(0,1,220), np.random.normal(4,0.8,80)])
for bins in [8, 30]:
    plt.figure(); plt.hist(x, bins=bins, density=True, color="steelblue", alpha=0.5,
        edgecolor="white")
    plt.title(f"Histogram bins={bins}"); plt.savefig(f"hist{bins}.pdf")
# KDE with two bandwidths
from scipy.stats import gaussian_kde
for bw in [0.2, 1.0]:
    kde = gaussian_kde(x, bw_method=bw)
    xs = np.linspace(-3, 7, 400)
    plt.figure(); plt.plot(xs, kde(xs), color="orangered", lw=2)
    plt.title(f"KDE bw={bw}"); plt.savefig(f"kde{bw}.pdf")
```

Listing 4.2: Brushing and linking with Altair selections.

```
import altair as alt, pandas as pd, numpy as np
np.random.seed(1)
df = pd.DataFrame({"price": np.random.lognormal(3,0.5,300),
                   "mileage": np.random.normal(50,15,300)})
brush = alt.selection_interval(name="brush")
scatter = alt.Chart(df).mark_point().encode(
    x="mileage:Q", y="price:Q", color=alt.condition(brush, alt.value("orangered"), alt.
        value("steelblue"))
).add_params(brush).properties(width=260, height=180)
hist = alt.Chart(df).mark_bar().encode(
    x=alt.X("price:Q", bin=alt.Bin(maxbins=20)), y="count()",
    color=alt.condition(brush, alt.value("orangered"), alt.value("lightgray"))
).transform_filter(brush).properties(width=260, height=100)
chart = alt.vconcat(scatter, hist)
chart  # in a notebook, drag on the scatterplot to see the linked histogram update
```

4.5 Chapter Notes

Cleveland (1993) and Wilkinson (Grammar of Graphics, 2005) grounded statistical charts; Scott and Freedman–Diaconis gave principled bin widths; Shneiderman (1996) articulated the mantra and dynamic queries; Becker and Cleveland's brushing (1987) and Buja et al. linking remain the interaction canon; Heer and Shneiderman (2012) surveyed interactive dynamics. The next chapter scales these ideas to networks.

4.6 Exercises

E4.1 **Bin rules.** For $n = 500$, $\sigma = 12$, IQR$= 10$, compute Sturges k, Scott h, and Freedman–Diaconis h. Convert h to k for range 0–100. Which rule is most robust to outliers and why?

E4.2 **Bandwidth trade-off.** Given a bimodal mixture $0.7N(0,1) + 0.3N(4,0.8)$, sketch KDEs at $h = 0.2, 0.6, 1.5$. Label where the second mode disappears (bias) and where wiggles appear (variance).

E4.3 **SPLOM vs parallel coordinates.** For a 6-variable dataset, list one question answered faster by a SPLOM and one faster by parallel coordinates. Propose axis ordering and opacity settings to reduce clutter in the parallel-coordinates view.

E4.4 **Build brushing.** Using Altair (or D3), implement the price vs mileage brushing in Listing 4.2 and add a second slider that filters by year. Report interaction latency strategy if the table grew to 10^6 rows.

E4.5 **Details on demand.** Design a tooltip and a side-panel details view for a scatterplot outlier. What attributes go in the tooltip (immediate) vs the panel (on click), and why? Justify with the mantra and with latency/accessibility.

Chapter 5

Graph & Network Visualisation

"Nodes where things are, edges what connects them." — But a hairball of tangled edges says nothing. Layout is the art of untangling.

From zero: we build here, no prior graph-drawing needed. We assume only nodes and edges. Every layout—force, hierarchy, treemap, matrix—is motivated by intuition, then defined, then exercised, with code. No prior network visualisation is required.

Learning Outcomes

After this chapter you will be able to:

(L1) compare node-link, matrix, and hierarchical encodings for network tasks;

(L2) explain force-directed, layered, and circular layouts and their aesthetics;

(L3) critique layouts by crossings, bends, angular resolution, and symmetry;

(L4) visualise trees and hierarchies via treemap, sunburst, and icicle;

(L5) encode multivariate attributes on nodes and edges and handle large graphs.

5.1 The Problem: From Hairball to Insight

Intuition. Draw 50 nodes with 200 random edges as dots and straight lines: you get a hairball where crossings hide every cluster. The same graph drawn with a matrix (rows and columns are nodes, cells are edges) shows clusters as blocks. No encoding is best for all questions; each hides one truth to reveal another.

Formal view. A graph $G = (V, E)$ with $|V| = n$, $|E| = m$. A *layout* assigns positions $p_v \in \mathbb{R}^2$ (or matrix cells) to vertices and routes edges as curves. Tasks drive choice: topology tasks ("are A and B connected?",

"what clusters exist?") favour node-link with good layout; adjacency/comparison tasks ("which edges exist for node v?") favour matrix; hierarchy tasks favour containment.

Definition 5.1 (Aesthetics)**.** Common aesthetics to optimise (often conflicting): minimise edge crossings and bends, maximise angular resolution at nodes and symmetry, keep edge lengths uniform, respect grouping (clusters together), and keep aspect ratio near 1. No layout optimises all; force-directed trades crossings for symmetry; layered trades edge length for hierarchy clarity.

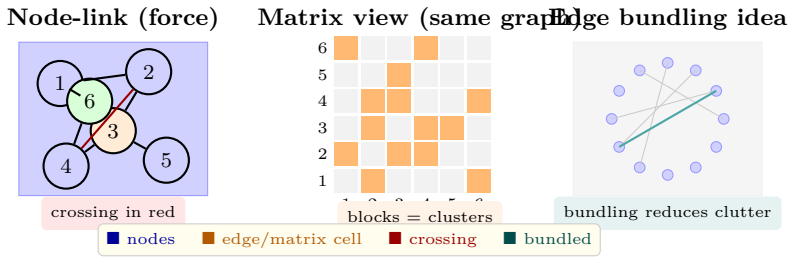

Figure 5.1: Same 6-node graph as node-link (crossing highlighted) vs matrix (clusters as blocks) vs edge-bundling concept (centre routed). Choice depends on task.

5.2 Layout Families

Intuition. Springs want short edges; charges want nodes apart. A force-directed layout simulates both until equilibrium—clusters emerge because intra-cluster springs are many and inter-cluster springs are few. A layered layout (Sugiyama) instead respects direction: put sources at the top, sinks at the bottom, minimise crossings layer by layer.

Definition 5.2 (Force-directed and Sugiyama)**.** *Fruchterman–Reingold*: repulsion $f_r \propto 1/d^2$ between all nodes, attraction $f_a \propto d$ along edges, iterate until cooling. *Sugiyama framework*: (1) cycle removal, (2) layer assignment (longest path), (3) crossing reduction via barycentre ordering, (4) coordinate assignment. *Circular/hive*: nodes on circle or parallel axes; edges as arcs.

For large graphs ($n > 10^4$), draw summaries: sample, filter by degree, aggregate clusters into metanodes, or switch to matrix. Edge bundling (force-directed or hierarchical) merges nearby edges to reduce hairball while preserving coarse connectivity.

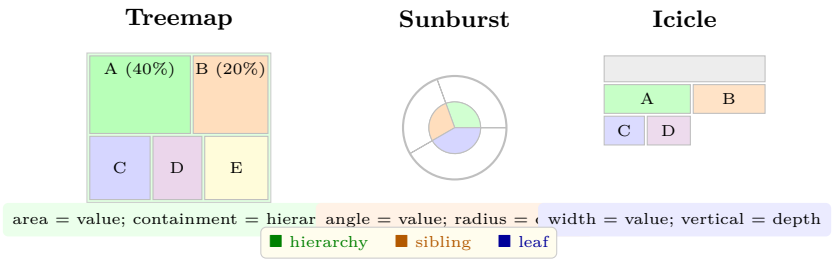

Figure 5.2: Hierarchy encodings: treemap (nested rectangles), sunburst (radial), icicle (stacked). All use containment or adjacency to show parent–child and area/angle to show value.

Definition 5.3 (Hierarchy encodings)**.** For a tree with values on leaves: *treemap* recursively slices a rectangle proportional to value (squarified for aspect ratio). *Sunburst* maps depth to radius and value to angle. *Icicle*

stacks layers vertically, width proportional to value. Choice trades space efficiency (treemap) vs path readability (sunburst/icicle).

5.3 Choosing Between Node-Link and Matrix

Use node-link when paths and topology matter and the graph is sparse ($m \ll n^2$). Use matrix when adjacency lookup, cluster Blocks, or dense graphs matter; matrices scale to thousands of nodes where node-link collapses. Hybrid views (NodeTrix) show clusters as matrices inside a node-link overview, recruiting both strengths. For temporal networks, animate with care—small multiples of time slices often beat animation for comparison tasks.

5.4 Multivariate Networks and Scale

Intuition. Real networks carry attributes: people have age and role, edges have weight and time. Encode these on top of layout with colour, size, thickness, dash, and label—but only two or three at once, or the hairball returns in colour.

Example 5.1 (Ordering a matrix)**.** A 20-node friendship matrix looks random in input order. Reorder rows/columns by community (Louvain) and blocks appear on the diagonal. Ordering is the matrix analogue of force layout—both reveal clusters; one via position, the other via permutation.

For scale, use *semantic zoom*: at overview, show clusters as metanodes; on zoom, expand. Filter by degree or centrality; sample edges by weight; or aggregate time into slices with small multiples.

Listing 5.1: Force-directed layout and matrix ordering with NetworkX.

```python
import networkx as nx, matplotlib.pyplot as plt, numpy as np

G = nx.karate_club_graph()
pos = nx.spring_layout(G, seed=0, k=0.35, iterations=80)  # Fruchterman--Reingold
plt.figure(figsize=(4,3)); nx.draw(G, pos, node_color="steelblue", edge_color="gray",
    with_labels=False, node_size=90)
plt.title("Force-directed (karate club)"); plt.savefig("force.pdf")

# Matrix reordered by degree
order = sorted(G.nodes(), key=lambda n: G.degree[n], reverse=True)
A = nx.to_numpy_array(G, nodelist=order)
plt.figure(figsize=(3,3)); plt.imshow(A, cmap="Oranges", interpolation="nearest")
plt.title("Adjacency matrix (degree order)"); plt.savefig("matrix.pdf")
```

Listing 5.2: D3 force simulation sketch (JavaScript).

```javascript
// D3 force: nodes repel, links attract, centre gravity
const simulation = d3.forceSimulation(nodes)
  .force("link", d3.forceLink(links).id(d=>d.id).distance(40))
  .force("charge", d3.forceManyBody().strength(-120))
  .force("center", d3.forceCenter(width/2, height/2))
  .force("collision", d3.forceCollide().radius(12));
simulation.on("tick", () => {
```

```
  link.attr("x1",d=>d.source.x).attr("y1",d=>d.source.y)
      .attr("x2",d=>d.target.x).attr("y2",d=>d.target.y);
  node.attr("cx",d=>d.x).attr("cy",d=>d.y);
});
```

5.5 Chapter Notes

Force layouts from Eades (1984) and Fruchterman–Reingold (1991); Sugiyama layered framework (1981); treemaps from Shneiderman (1992); matrix views systematised by Ghoniem et al. (2004) showing matrices beat node-link for dense graphs; edge bundling by Holten (2006); hierarchy surveys by Schulz (2011). NodeTrix hybrids by Henry et al. The next chapter leaves discrete networks for continuous fields and volumes.

5.6 Exercises

E5.1 **Draw and count.** For the 6-node graph in Fig. 5.1, draw a force-directed sketch and a matrix view. Count edge crossings in each and state which task (path vs adjacency) each supports.

E5.2 **Sugiyama steps.** Given a 7-node DAG with edges $1\to\{2,3\}, 2\to4, 3\to4, 4\to5, 5\to6$, assign layers by longest path, order each layer by barycentre, and sketch the final layered drawing. How many crossings remain?

E5.3 **Treemap vs sunburst.** For hierarchy root(100) with children A(40), B(30), C(20), D(10) where A has leaves A1(25),A2(15), compute rectangle areas (treemap) and angles (sunburst). Which encoding makes it easier to compare A1 vs D?

E5.4 **Matrix ordering.** Take a 12-node caveman graph (3 cliques of 4 with one inter-clique edge each). Show its matrix in random vs Louvain order; describe the block pattern and why ordering matters more as density grows.

E5.5 **D3 tuning.** In Listing 5.2, vary `charge` from -30 to -300 and `distance` from 20 to 100. Describe the effect on cluster separation and edge length uniformity; when does the layout collapse or explode?

Chapter 6

Volume & Scientific Visualisation

"A CT scan is a stack of shadows; visualisation turns shadows into substance." — From medical volumes to fluid flows, science measures fields, not just tables. This chapter shows how to see them.

From zero: we build here, no prior scientific visualisation needed. We start from what a scalar or vector field is, then derive isosurfaces, volume rendering, glyphs, and streamlines, with acceleration and code sketches. No prior graphics is assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) distinguish scalar, vector, and tensor fields and their sampling on grids;

(L2) extract isosurfaces via marching cubes and choose isovalues;

(L3) design transfer functions and explain direct volume rendering by ray casting;

(L4) visualise vector fields via glyphs, streamlines, and line integral convolution;

(L5) map scientific domains (medical, flow, geospatial) to appropriate methods and optimise via acceleration.

6.1 Fields on Grids

Intuition. Temperature over a room varies at every point—it is a field. A CT scan samples density at voxels (3-D pixels) on a regular grid. A wind simulation samples velocity vectors at grid points. Visualisation must reconstruct the continuous field from these samples and show structure within it.

Formal view. A *scalar field* $f : \mathbb{R}^3 \to \mathbb{R}$ (pressure, density); a *vector field* $\mathbf{v} : \mathbb{R}^3 \to \mathbb{R}^3$ (velocity, force); a *tensor field* maps to matrices (stress, diffusion). Data are *sampled* on grids: regular (voxels), rectilinear,

structured curvilinear, or unstructured (tetrahedra). Reconstruction uses interpolation (trilinear, higher-order); filtering smooths; gradient ∇f enables shading.

Definition 6.1 (Grid types)**.** *Voxel grid*: regular $n_x \times n_y \times n_z$ with spacing h. *Cell*: the cube between 8 neighbours; its value is interpolated. *Scattered*: points with no regular connectivity; triangulation required. Medical CT/MRI and simulation outputs are typically regular grids; geospatial may be terrain + layers.

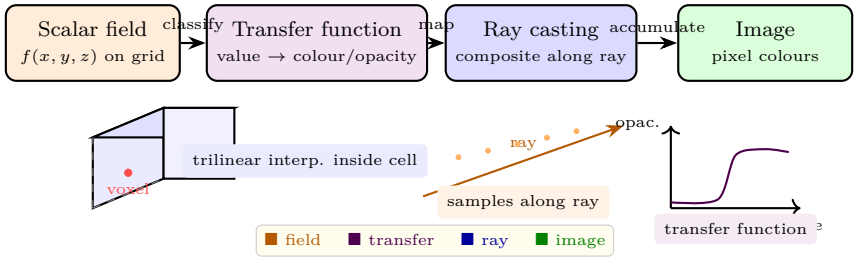

Figure 6.1: Volume rendering pipeline (top): field → transfer function → ray casting → image. Bottom: voxel/cell with trilinear interpolation, sampling along a ray, and an opacity transfer function.

6.2 Scalar Fields: Isosurfaces and Volume Rendering

Intuition. An isosurface is the 3-D equivalent of a contour line: the set $\{x \mid f(x) = k\}$ for chosen isovalue k. Pick k to be skin vs bone in CT and the surface appears. Volume rendering instead shows the whole volume with translucency: bone opaque, flesh semi-transparent, air invisible—controlled by the transfer function.

Definition 6.2 (Marching cubes and ray casting)**.** *Marching cubes*: for each cube of 8 voxels, classify each vertex as inside ($f > k$) or outside; use a 256-case table (15 unique) to emit triangles interpolating edge intersections. *Direct volume rendering*: for each pixel cast a ray, sample f at steps, map via transfer function to colour/opacity (c, α), composite front-to-back: $C = \sum c_i \alpha_i \prod_{j<i}(1 - \alpha_j)$.

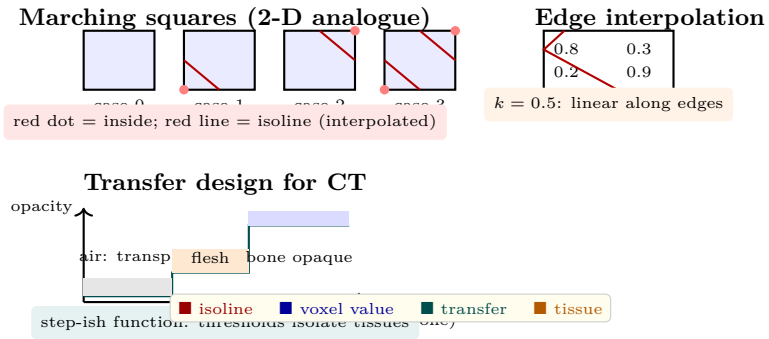

Figure 6.2: Scalar fields: 2-D marching squares cases (top left), edge interpolation for isovalue k (top right), and a CT transfer function mapping density to opacity (bottom). Marching cubes extends this to 8-vertex cubes with triangles.

Transfer design is the craft: too narrow a window and you hide anatomy; too wide and everything is translucent fog. Histogram of voxel values plus gradient magnitude guides placement of opacity ramps at material boundaries.

6.3 Vector and Tensor Fields

Intuition. Wind at each point has direction and speed—an arrow. But thousands of arrows occlude; streamlines that follow the wind tell the story more cleanly, and a texture smeared along the flow (LIC) shows it everywhere at once.

Definition 6.3 (Glyphs, streamlines, LIC)**.** *Glyph*: an icon (arrow, ellipsoid) at sampled points encoding vector magnitude/direction (and tensor shape). *Streamline*: curve $x(t)$ with $dx/dt = \mathbf{v}(x)$ from a seed, integrated via Euler or Runge–Kutta; streamsurface for dense seeds; streamsurface/pillars for 3-D. *Line integral convolution (LIC)*: convolve noise texture along streamlines to produce streaks coherent with flow, revealing topology without seeding.

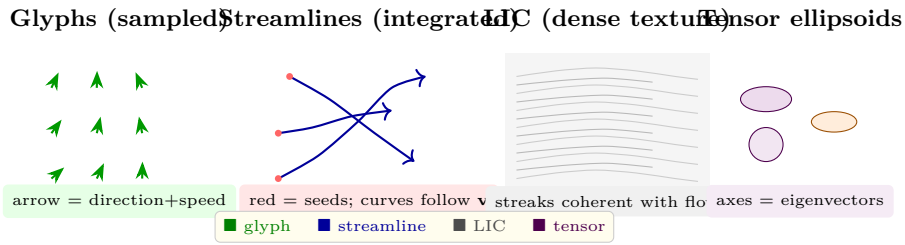

Figure 6.3: Vector field views: glyphs at samples (left), streamlines from seeds (centre), LIC dense texture (centre-right), and tensor ellipsoids (right). Density and occlusion trade off across methods.

Tensors (diffusion MRI, stress) add shape: an ellipsoid's axes are eigenvectors, lengths are eigenvalues; isotropy vs anisotropy maps to fractional anisotropy colour (red x, green y, blue z).

6.4 Domains, Projections, and Acceleration

Domains: *medical* (CT/MRI iso + volume), *flow* (CFD streamlines/LIC), *geospatial* (terrain + fields + map projections: Mercator, equal-area), *time-varying* (animation or space-time cube). Acceleration: *octrees/bricks* skip empty space, *early ray termination* stops when opacity saturates, *level-of-detail* and *GPU ray casting* exploit parallelism.

Listing 6.1: Marching cubes isosurface with scikit-image.

```
import numpy as np
from skimage import measure
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

# Synthetic scalar field: two blobs
x,y,z = np.ogrid[-3:3:40j, -3:3:40j, -3:3:40j]
field = np.exp(-((x-1)**2+y**2+z**2)) + 0.7*np.exp(-((x+1)**2+y**2+z**2))
verts, faces, _, _ = measure.marching_cubes(field, level=0.5)  # isovalue
fig = plt.figure(); ax = fig.add_subplot(111, projection="3d")
mesh = Poly3DCollection(verts[faces], alpha=0.7, edgecolor="none", facecolor="steelblue")
ax.add_collection3d(mesh); ax.set_xlim(0,40); ax.set_ylim(0,40); ax.set_zlim(0,40)
plt.title("Isosurface at k=0.5"); plt.savefig("isosurface.pdf")
```

Listing 6.2: Volume ray-marching sketch (front-to-back compositing).

```python
import numpy as np

def ray_march(field, ray_origin, ray_dir, transfer, steps=128, dt=0.05):
    """field: 3-D array; transfer: value -> (r,g,b,alpha); returns pixel colour."""
    pos = np.array(ray_origin, float); colour = np.zeros(3); alpha_acc = 0.0
    for _ in range(steps):
        v = trilinear(field, pos)          # interpolate
        r,g,b,a = transfer(v)
        # front-to-back compositing
        colour += (1-alpha_acc) * a * np.array([r,g,b])
        alpha_acc += (1-alpha_acc) * a
        if alpha_acc >= 0.99: break        # early termination
        pos += dt * np.array(ray_dir)
    return colour

def trilinear(vol, p):
    # placeholder: clamp and trilinearly interpolate vol at continuous p
    return float(np.clip(vol[int(p[0])%vol.shape[0], int(p[1])%vol.shape[1], int(p[2])%vol
        .shape[2]], 0, 1))

def transfer_ct(v):
    # toy CT: air transparent, flesh pink semi, bone white opaque
    if v < 0.3: return (0,0,0,0.0)
    elif v < 0.6: return (0.9,0.6,0.6,0.2)
    else: return (1,1,1,0.9)
```

6.5 Chapter Notes

Lorensen and Cline (1987) introduced marching cubes; Levoy (1988) and Drebin et al. formulated volume rendering; Cabral and Leedom (1993) gave LIC; map projections date to Mercator (1569) with modern equal-area alternatives; acceleration via octrees and GPU ray casting surveyed by Engel et al. The next chapter asks whether these pictures actually help humans think.

6.6 Exercises

E6.1 **Isocontour by hand.** On a 2×2 grid with values $0.2, 0.9$ (top) and $0.8, 0.3$ (bottom), extract the $k = 0.5$ isoline via marching squares. List edge intersections by linear interpolation and draw the line segments.

E6.2 **Transfer design.** Given a CT histogram with peaks at 0.1 (air), 0.5 (soft tissue), 0.85 (bone), sketch an opacity transfer function that isolates each tissue. Explain the effect of widening the bone window.

E6.3 **Streamline integration.** For $\mathbf{v}(x, y) = (-y, x)$ (rotation), trace a streamline from $(1, 0)$ using Euler with $dt = 0.2$ for 5 steps and compare to the true circle. Why does Euler spiral outward?

E6.4 **LIC vs glyphs.** For a turbulent field with 10k vectors, argue when LIC beats glyphs and when glyphs beat LIC, considering occlusion, seed dependence, and ability to read magnitude.

E6.5 **Octree culling.** A 512^3 volume is 70% empty (value $<$ threshold). Estimate memory and ray-step savings from an octree that culls empty bricks of 32^3. When does early ray termination give additional gain?

Chapter 7

Perceptual, Cognitive & Evaluation

"A visualisation is a hypothesis that a picture helps." — Hypotheses need tests. This chapter asks how to know whether a picture actually helps and how the mind works with it.

From zero: we build here, no prior perception or statistics needed. We start from what eyes do in milliseconds and what memory holds in seconds, then build controlled studies, measures, and analyses, with examples. No prior evaluation experience is required.

Learning Outcomes

After this chapter you will be able to:

(L1) explain pre-attentive pop-out, change blindness, and attention limits;

(L2) describe mental models, chunking, cognitive load, and dual-process reasoning;

(L3) design a controlled study: hypothesis, variables, tasks, and protocol;

(L4) choose measures (accuracy, time, preference, insight) and avoid pitfalls;

(L5) analyse results with confidence intervals, t-tests/ANOVA, and effect sizes.

7.1 Beyond the Retina: Attention and Memory

Intuition. You can spot a red dot instantly, but try finding a red vertical line among red horizontal and blue vertical lines—now you must search deliberately, and if the display flickers you may miss a large change entirely. Seeing is not just optics; it is attention and memory.

Formal view. *Pre-attentive* features (hue, size, orientation, motion) are processed in parallel (<200 ms, pop-out). *Conjunctions* require serial focal attention. *Change blindness*: large changes go unseen without focused

attention, especially during saccades or flicker. *Attention* is a limited spotlight; *working memory* holds ∼4 chunks; *long-term memory* stores schemas that chunk complex patterns into one unit.

Definition 7.1 (Cognitive concepts)**.** *Mental model*: internal representation of data structure built from the picture. *Chunking*: grouping pieces into a unit (e.g. a cluster of dots becomes "the high group"). *Cognitive load*: effort to hold and manipulate chunks; extraneous load from poor design harms insight. *Dual-process*: System 1 (fast, pattern-matched) vs System 2 (slow, analytical); visualisation aims to make correct insights System 1-fast.

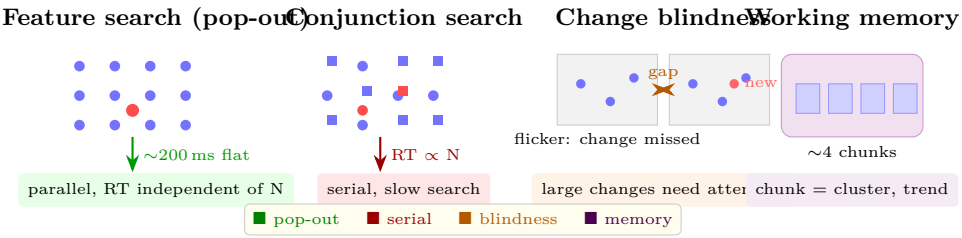

Figure 7.1: Perception and attention: feature search is parallel (left); conjunction search is serial (centre-left); change blindness hides large changes without attention (centre-right); working memory holds ∼4 chunks (right).

Good design respects these limits: encode the answer in a pre-attentive feature when possible, keep the number of colours and groups small, and avoid flicker or gratuitous animation that induces blindness.

7.2 Designing an Evaluation

Intuition. Claiming "my chart is better" without a test is like claiming a drug works without a trial. A controlled study makes the claim falsifiable: fix a task, vary only the picture, measure what matters.

Definition 7.2 (Study anatomy)**.** *Research question* → *hypothesis* (e.g. "position beats angle for ratio judgment"). *Independent variable* (IV): what you vary (encoding). *Dependent variable* (DV): what you measure (accuracy, time, preference, insight). *Controls*: keep data, task, and participants balanced. *Protocol*: within-subjects (each participant sees all conditions, counterbalanced) vs between-subjects; randomised order; training and practice trials; ethics consent.

Classic tasks: Cleveland–McGill ratio judgment ("what percent is the smaller of the larger?"), Heer–Bostock replication with crowdsourcing, insight diaries for open-ended exploration. Choose tasks that span the taxonomy: value retrieval, comparison, trend, outlier, distribution.

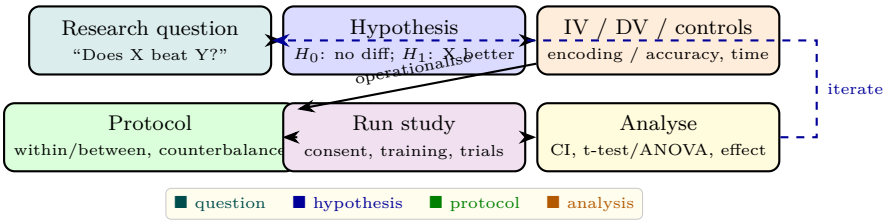

Figure 7.2: Evaluation pipeline: from question to hypothesis to variables to protocol to analysis. Iteration is expected; pilot first.

Within-subjects is powerful (each person is their own control) but needs counterbalancing and washout; between-subjects avoids carryover but needs more participants. Pilot with 3–5 colleagues to catch instruction and timing bugs before recruiting 20–30 participants.

7.3 Measures and Analysis

Measures: *accuracy* (absolute error, log error), *time* (from stimulus to answer, often log-transformed), *preference* (Likert, ranking), *insight* (count, quality via coding), *confidence* (self-report). Avoid measuring only preference—people prefer what they know, not what is accurate.

Analysis: plot data with means and 95% confidence intervals (CIs) before any test. For two conditions, a paired t-test (within) or two-sample t-test (between); for > 2, ANOVA. Report *effect size* (Cohen's d for means, r for correlations): a $p < 0.05$ with $d = 0.1$ is statistically but not practically significant. Power analysis before running: for $d = 0.5$, $\alpha = 0.05$, power 0.8, need ∼34 participants within-subjects.

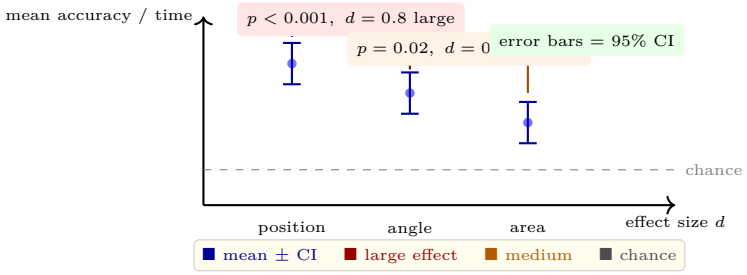

Figure 7.3: Results as means with 95% confidence intervals. Non-overlapping CIs signal a difference; report p-values and effect sizes (d), not just significance stars.

Example 7.1 (Heer–Bostock replication sketch)**.** Recruit 30 participants on a crowdsourcing platform. Within-subjects, each judges 20 ratio pairs per encoding (position, length, angle, area) in random order. DV: log absolute error $|\log(\text{judged}/\text{true})|$. Pilot shows ∼8 s per trial. Analysis: mean log error per encoding with 95% CI, repeated-measures ANOVA, post-hoc paired t-tests with Bonferroni, and Cohen's d.

Listing 7.1: Analysing a two-condition study with CIs and t-test.

```
import numpy as np, scipy.stats as st, matplotlib.pyplot as plt

# synthetic: position more accurate than angle
np.random.seed(0)
pos = np.random.normal(0.12, 0.06, 30)   # log absolute error
ang = np.random.normal(0.22, 0.08, 30)
for name, arr in [("position",pos), ("angle",ang)]:
    m, se = arr.mean(), st.sem(arr)
    ci = se * st.t.ppf(0.975, len(arr)-1)
    print(f"{name}: mean={m:.3f} 95% CI [{m-ci:.3f},{m+ci:.3f}]")

t, p = st.ttest_rel(pos, ang)            # within-subjects
d = (pos-ang).mean() / (pos-ang).std(ddof=1)  # Cohen d (paired)
print(f"paired t={t:.2f} p={p:.4f} d={d:.2f}")

# plot means with CI
plt.figure(figsize=(4,2.5)); plt.errorbar([0,1],[pos.mean(),ang.mean()],
```

```
  yerr=[st.sem(pos)*st.t.ppf(0.975,29), st.sem(ang)*st.t.ppf(0.975,29)], fmt="o", capsize
      =6, color="steelblue")
plt.xticks([0,1],["position","angle"]); plt.ylabel("log absolute error"); plt.title("Means
    with 95% CI")
plt.tight_layout(); plt.savefig("ci.pdf")
```

Listing 7.2: Power sketch and preference vs accuracy check.

```
import statsmodels.stats.power as smp

# how many participants for d=0.5, alpha=0.05, power=0.8 (paired)?
analysis = smp.TTestPower()
n = analysis.solve_power(effect_size=0.5, alpha=0.05, power=0.8, alternative="two-sided")
print(f"needed n ~ {n:.0f} per group (paired, so total N similar)")

# preference vs accuracy: people may prefer angle (familiar pie) but perform worse
# always report both; a scatterplot of preference rank vs error reveals the dissociation
```

7.4 Qualitative and Insight-Based Evaluation

Not every question is a ratio judgment. For exploration, use *insight diaries*, *think-aloud*, and *milestone* coding; for communication, test *comprehension* and *recall* after a delay. Qualitative data are coded by two raters with Cohen's κ for agreement. Mixed methods—numbers plus quotes—often tell the full story.

7.5 Chapter Notes

Pre-attentive and attention foundations from Treisman and Wolfe; change blindness from Simons and Levin; cognitive load from Sweller; Cleveland–McGill (1984) and Heer–Bostock (2010) set the evaluation standard for channels; Munzner (2014) framed evaluation across nested levels; Lam et al. (2012) catalogued seven evaluation scenarios. The final chapter turns evaluation into production.

7.6 Exercises

E7.1 **Pop-out design.** Design two searches on the same 30-item grid: one that pops out via colour alone and one that requires conjunction (colour+shape). Predict reaction time vs set size (4,16,30) for each and sketch the expected lines.

E7.2 **Study design.** Propose a within-subjects study comparing a rainbow vs a sequential palette for quantitative reading. State H_0/H_1, IV/DV, controls, number of trials, counterbalancing, and one threat (learning effect) with mitigation.

E7.3 **Analyse given data.** Two encodings produce log errors: A $[0.10, 0.14, 0.11, 0.18, 0.13]$ and B $[0.22, 0.19, 0.25, 0.21, 0.20]$. Compute means, 95% CIs, paired t, and Cohen's d. Is the difference practically significant?

E7.4 **Power planning.** For a between-subjects comparison expecting $d = 0.4$, $\alpha = 0.05$, power 0.8, estimate required N per group. How does switching to within-subjects change N, and what new threat appears?

E7.5 **Insight diary.** Design an insight-based protocol for a visual analytics tool: task prompt, think-aloud instructions, insight coding rubric, and inter-rater reliability plan. When would this beat a timed accuracy study?

Chapter 8

Tools & Storytelling

"Data without a story is noise; a story without data is anecdote." — Tools turn encodings into pictures; narrative turns pictures into arguments that move decisions.

From zero: we build here, no prior tool or storytelling training needed. We introduce the modern stack—D3.js, Vega-Lite, Tableau, Python—from intuition, then build layered specs, data joins, and narrative structures, with code that runs. No prior web or design background is required.

Learning Outcomes

After this chapter you will be able to:

(L1) contrast imperative (D3) vs declarative (Vega-Lite) vs GUI (Tableau) tool models;

(L2) write a Vega-Lite spec: data, transform, mark, encoding, view composition;

(L3) script a D3 data join with scales, axes, and transitions;

(L4) structure a data narrative: martini-glass, drill-down, scrollytelling, and dashboard;

(L5) deliver an end-to-end story from dataset to sketches to interactive presentation.

8.1 The Tool Landscape

Intuition. You can hammer every nail by hand (D3: full control, full code), use a nail gun with templates (Vega-Lite: declare what you want, the engine decides how), or buy pre-hung doors (Tableau: drag, drop, publish). Control trades against speed and scaffolding. No tool wins for all tasks; professionals use at least two.

Definition 8.1 (Tool spectrum)**.** *D3.js*: low-level JavaScript library binding data to DOM/SVG, explicit scales and transitions, maximal expressiveness. *Vega* and *Vega-Lite*: declarative grammars; Vega-Lite is a higher-level JSON grammar compiling to Vega, with automatic scales/axes/legends and view composition (layer, concat, facet). *Tableau/Power BI*: GUI shelf model (rows/columns/marks), rapid exploration, limited custom

encodings. *Python stack*: Matplotlib (imperative, publication), Altair (Python Vega-Lite), Plotly (interactive, WebGL).

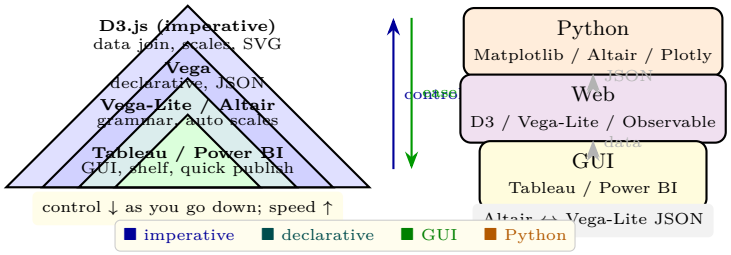

Figure 8.1: Tool pyramid: D3 at the top (most control), Vega-Lite/Altair in the middle (grammar), Tableau at the base (GUI speed). Altair compiles Python to Vega-Lite JSON, bridging Python and the Web.

Rule of thumb: prototype in Altair/Tableau, refine in Vega-Lite, and drop to D3 only when you need a custom encoding or interaction Vega-Lite cannot express.

8.2 Vega-Lite: A Grammar of Interactive Graphics

Intuition. Instead of "draw a circle at (x, y)", say "map GDP to x, life expectancy to y, continent to colour, and use a point mark; facet by year." The grammar decides scales, axes, and legends. Composition then builds dashboards from simple views.

Definition 8.2 (Vega-Lite spec)**.** A spec is JSON with: `data` (values or URL), `transform` (filter, calculate, bin, aggregate), `mark` (point, bar, line, area, rect, arc), `encoding` (channel → field with type `Q/N/O/T` and scale), and `view composition`: `layer` (superpose), `hconcat/vconcat` (juxtapose), `facet` (small multiples), `concat` with `selection` for brushing/linking.

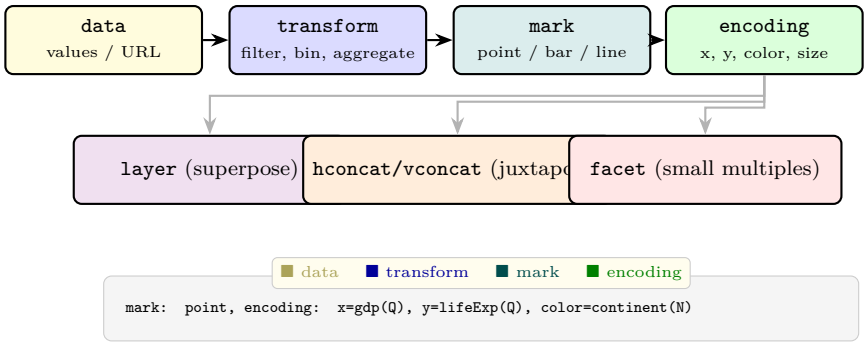

Figure 8.2: Vega-Lite spec anatomy: data → transform → mark → encoding, then composed by layer, concat, or facet. Bottom: a minimal point spec.

Selections are the interaction extension: a `selection_interval` bound to x or xy becomes a brush; `condition` ties colour or opacity to selection state; `transform_filter` links views.

8.3 D3.js: Data Joins, Scales, and Transitions

Intuition. D3 thinks in *data joins*: bind an array to DOM elements, handle entering, updating, and exiting elements, and let scales map data to pixels. You own every pixel—and every bug.

Definition 8.3 (D3 join)**.** `selection.data(data, key).join(enter, update, exit)` creates/updates/removes SVG elements per data item. *Scales* (`scaleLinear, scaleBand, scaleOrdinal`) map domain → range; *axes* (`axisBottom`) render ticks; *transitions* interpolate attributes over time. The general update pattern—data → join → scales → axes → transition—powers most D3 charts.

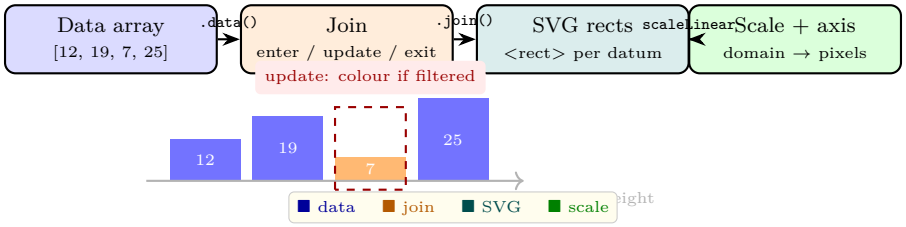

Figure 8.3: D3 data join: data → join (enter/update/exit) → SVG elements → scales. Changing data updates bars via the join, not by redrawing everything.

When Vega-Lite suffices, prefer it; when you need a bespoke force layout, hierarchical edge bundling, or scrollytelling step control, D3 repays its cost.

8.4 Storytelling: From Exploration to Narrative

Intuition. An exploratory dashboard lets readers ask any question; a narrative answers one question well, guiding attention step by step. The best pieces do both: an author-driven opening that makes the point, then a reader-driven coda where the audience explores.

Definition 8.4 (Narrative structures)**.** *Martini-glass*: author-driven intro (narrowing) → reader-driven exploration (broad). *Interactive slideshow*: discrete steps with forward/back, each step a filtered view with annotation. *Drill-down*: overview → detail on demand via clicks. *Scrollytelling*: scroll position drives view transitions. All rely on *annotation* (labels, arrows, shaded regions) and *sequencing* (order matters more than any single view).

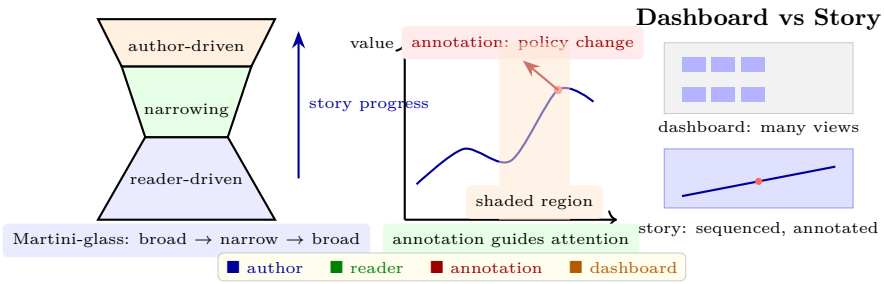

Figure 8.4: Narrative structures: martini-glass (left), annotation guiding a time-series (centre), and dashboard (overview) vs story (sequence) contrast (right).

Practical recipe: start with sketches on paper (2–3 alternatives), choose the narrative arc, annotate the one view that carries the main claim, and test on one colleague whether they get the point in 30 seconds.

Listing 8.1: Altair/Vega-Lite: layered, faceted, and brushed specs.

```
import altair as alt, pandas as pd, numpy as np
np.random.seed(2)
df = pd.DataFrame({"year": np.repeat([2019,2020,2021], 80),
```

```
                    "value": np.random.randn(240).cumsum(),
                    "group": np.tile(["A","B"], 120)})
# Layered: line + annotated rule
line = alt.Chart(df).mark_line().encode(x="year:O", y="mean(value):Q", color="group:N")
rule = alt.Chart(pd.DataFrame({"year":[2020]})).mark_rule(color="red", strokeDash=[4,4]).
    encode(x="year:O")
chart_layer = line + rule  # layer
# Facet: small multiples per group
chart_facet = alt.Chart(df).mark_point().encode(x="value:Q", y="value:Q").facet(column="
    group:N")
# Brush linking (as in Ch 4)
brush = alt.selection_interval(encodings=["x"])
base = alt.Chart(df).mark_point().encode(x="value:Q", y="value:Q", color=alt.condition(
    brush, alt.value("steelblue"), alt.value("lightgray"))).add_params(brush)
chart_layer  # renders in notebook; .to_json() for Vega-Lite
```

Listing 8.2: D3 data join with transition (JavaScript).

```
// General update pattern: data -> join -> transition
const svg = d3.select("svg"), x = d3.scaleBand().padding(0.1), y = d3.scaleLinear();
function update(data) {
  x.domain(data.map(d=>d.name)).range([0, width]);
  y.domain([0, d3.max(data,d=>d.value)]).range([height, 0]);
  svg.selectAll("rect").data(data, d=>d.name)
    .join(
      enter => enter.append("rect").attr("y", height).attr("height", 0),
      update => update,
      exit => exit.transition().duration(400).attr("y", height).attr("height", 0).remove()
    )
    .transition().duration(600)
    .attr("x", d=>x(d.name)).attr("y", d=>y(d.value))
    .attr("width", x.bandwidth()).attr("height", d=>height - y(d.value))
    .attr("fill", "steelblue");
  svg.select(".x-axis").call(d3.axisBottom(x));
  svg.select(".y-axis").call(d3.axisLeft(y));
}
update([{name:"A",value:12},{name:"B",value:19},{name:"C",value:7}]);
```

8.5 Capstone: From Dataset to Story

Pick a dataset you care about (e.g. gapminder, COVID, or a personal log). Sketch 3 story arcs, choose one, prototype 2 views in Altair, add one annotation that states the takeaway, and present as a 3-minute scrollytelling or slide. Evaluation: can a peer state your claim, the evidence, and one limitation after viewing?

8.6 Chapter Notes

Munzner (2014) framed the tool–task match; Heer et al. created D3 (2011) and Vega/Vega-Lite (2016–17); Satyanarayan et al. formalised the grammar and composition; Tableau's shelf model derives from Polaris (Stolte

et al. 2002); narrative structures from Segel and Heer (2010) and Kosara and Mackinlay (2013); scrollytelling popularised by the New York Times. This closes the 0→100 arc: perceive → encode → design → interact → connect → render → validate → narrate.

8.7 Exercises

E8.1 **Vega-Lite spec.** Write a Vega-Lite JSON spec for a layered chart: bars for mean value per continent with error bars (±1 sd) and a red rule at the global mean. Include `transform` for aggregation and `layer` for composition.

E8.2 **D3 join.** Using the update pattern in Listing 8.2, implement adding, updating, and removing bars when data changes from $[12, 19, 7]$ to $[19, 7, 25, 10]$. Describe which bars enter, update, and exit and how the transition looks.

E8.3 **Tool choice memo.** For tasks (a) quick CEO dashboard, (b) custom hierarchical edge bundling paper figure, (c) reproducible lab report, recommend D3 vs Vega-Lite vs Tableau/Python and justify with control vs speed and reproducibility.

E8.4 **Martini-glass storyboard.** Take the gapminder dataset (GDP vs life expectancy over time). Sketch a 4-step martini-glass story: opening claim, narrowing evidence (two views), and reader-driven exploration. Annotate the key frame.

E8.5 **Capstone critique.** Build a 2-view interactive story in Altair or D3 on any dataset. Ask two peers to state your claim and one limitation after 60 seconds. Report where they succeeded or missed and revise one annotation or sequencing choice.